Data analysis and bio-statistics

Izan Flores Tortosa & Jordi Vivas Sakuta

Exercises for practice 6

All the exercises for this practice will be written in a single R markdown script file (.Rmd extension). Don't forget to use comments to indicate each exercise in the code and what you are trying to achieve within each section. Please, keep each exercise statement in your R script as a comment in order to make the code more readable.

The exercises in practice 6 will be carried out using the provided dataset in a CSV file. This dataset contains the measurements of a real experiment which evaluates the effect of two drugs: fenofibrate and niacin in type 2 diabetes patients. For a complete description of the experiment please refer to the paper: https://doi.org/10.1016/j.atherosclerosis.2014.12.006

The first step before starting the exercise consist in loading the dataset using the read.csv2() function. Here, we are not going to use the metadata columns and we will focus only on observations from groups "control" and "T2DM_basal". So, you must use the following code to load your data and discarded the unused columns (variables) and rows (observations).

```
#Edit this code chunk to successfully load the CSV data
df <- read.csv2("bdLOWHDL_metabolomica.csv", header = T, sep = ";", dec = ",")
df <- df[ df$grupo == "control" | df$grupo == "T2DM_basal" ,c(which(colnames(df)=="grupo"),27:ncol(df))]
```

1- PCA used to identify outliers

Start by adding a synthetic outlier to our data. Execute the following code to produce a randomly generated outlier observation.

```
#Adds a copy of row 1 at the end of the data matrix
df <- rbind(df, df[1,])

#Replace the contents of the last row with random shifts
df[nrow(df), -1] <- apply(df[,-1], 2, function(x){ 0.9*sample(rnorm(100, mean = max(x, na.rm = T), sd =

#Randomize rows
df <- df[sample(1:nrow(df), nrow(df)), ]
```

Now, the df variable contains our original data plus one outlier. Lets try to identify the outlier using PCA. The PCA algorithm only woks with numeric data (just numbers, no NA, no strings). Since our data contains NA's, the first step consists in treating them. A commonly used strategy is to replace NA values with the median of all the observation. So, modify the data frame to discard the NA values by replacing each NA by the median of the corresponding column.

```
#Write your code here
df<-as.data.frame(lapply(df,function(x){
  if(is.numeric(x)){
    x[is.na(x)]<-median(x,na.rm= TRUE)
  }
  return(x)
}))
```

Finally, we are ready to execute the PCA algorithm. R provides a PCA implementation in the prcomp() function. Use the following code:

```
pca <- prcomp(df[,-1], scale. =  T, center = T )
```

```
#Write your code here
pca <- prcomp(df[,-1], scale. =  T, center = T )
```

Note that we are not supplying the first column to the PCA because it contains the grouping. Note that we are using the scale. = T and center = T parameters. This is important since scaling and centering normalizes all variables to the same variance.

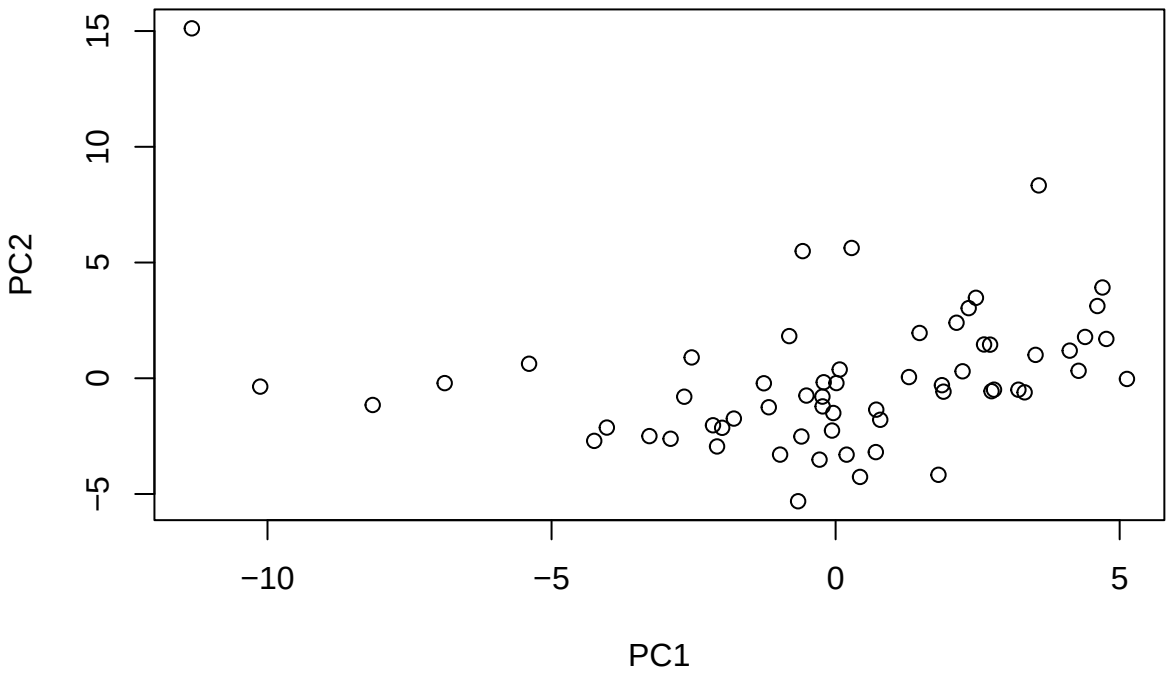
The returned prcomp() data is structured in a list with the following fields:

- x: The principal component (PC) matrix. Each PC is arranged in a column. The number of rows is the same as the number of observations.
- rotation: The rotation or loadings matrix. This matrix is the linear transformation applied to the data to obtain the PC matrix (x). This matrix is an square matrix since it relates original variables with PC's.
- Center: The calculated center of each variables.
- Scale : The applied scaling to each variable.
- sdev: The standard deviation of the PC's.

Use the PC matrix (pca$x) to produce an scatter plot of PC1 vs. PC2. Can you identify the outlier observation in this plot?

```
#Write your code here

plot(pca$x)
```



Now, write a few lines of R code to identify and delete the outlier row from the data. To do so, use the information provided by PC1 and PC2.

```
#Write your code here
media_x<- mean(pca$x[,"PC1"])
media_y<- mean(pca$x[,"PC2"])
d=sqrt((pca$x[,"PC1"]-media_x)^2+(pca$x[,"PC2"]-media_y)^2)

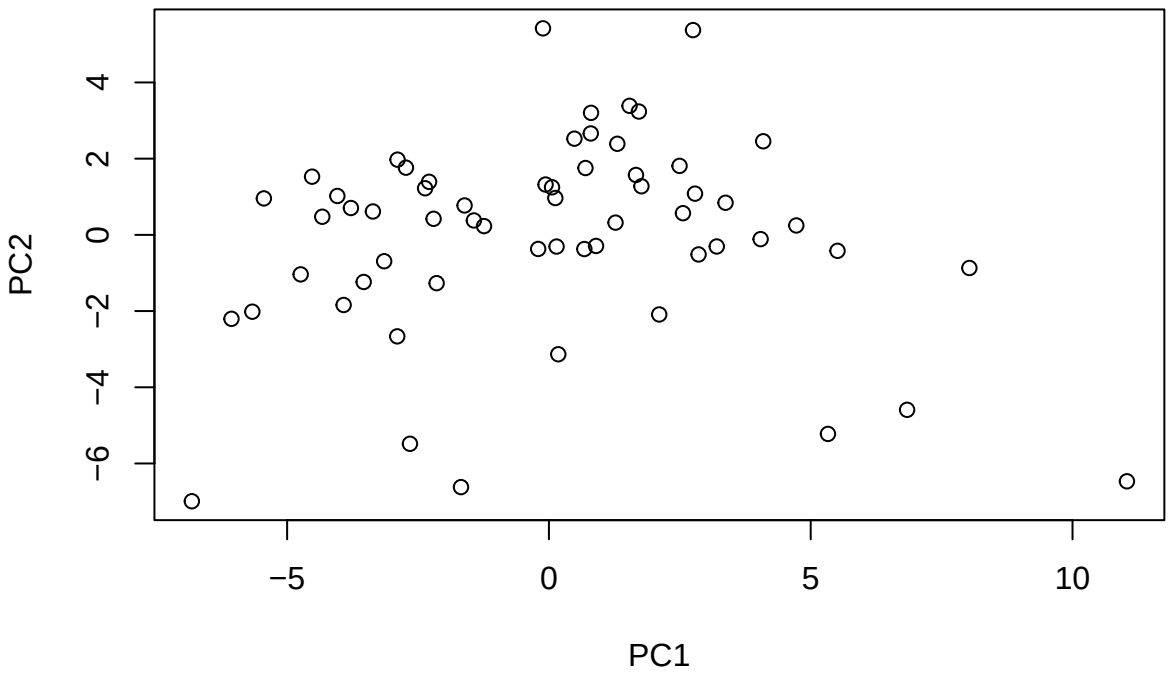
outlier<-which.max(d)

df1<-df[-outlier,]
```

Finally, repeat the PCA using the prcomp() function and produce the PC1 vs. PC2 scatter plot. Verify that the outlier has been successfully removed.

```
#Write your code here
pca1 <- prcomp(df1[,-1], scale. =  T, center = T )

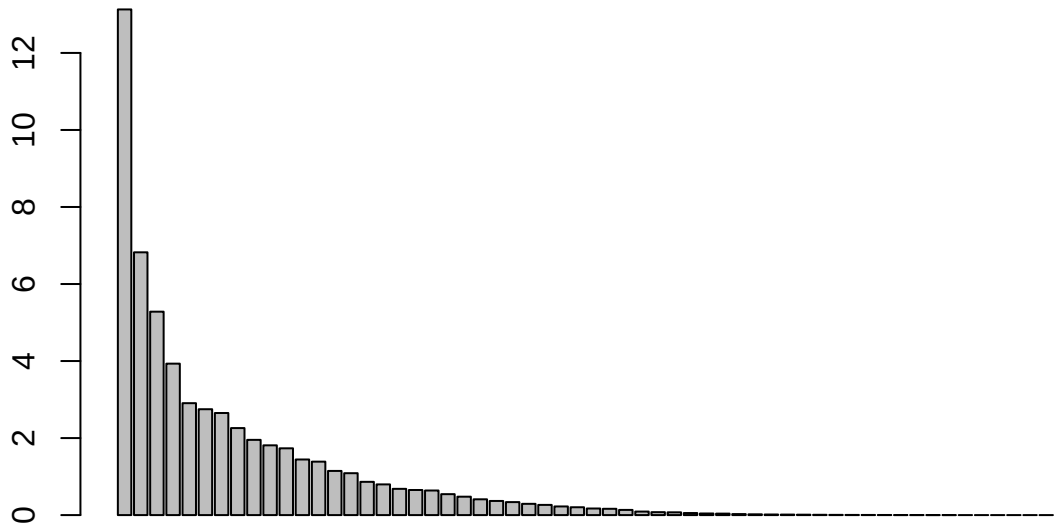
plot(pca1$x)
```



2-PCA as a dimensionality reduction tool

Since PCA finds the data projection which maximizes the variance in the first PC it can be used to reduce the dimensionality of the data. Use the pca's standard deviation (σ) to calculate the % of variance captured in each PC (remember that the variance is the σ^2). Then, produce a plot to display how the variance is distributed across all PC's (a bar chart is commonly used here).

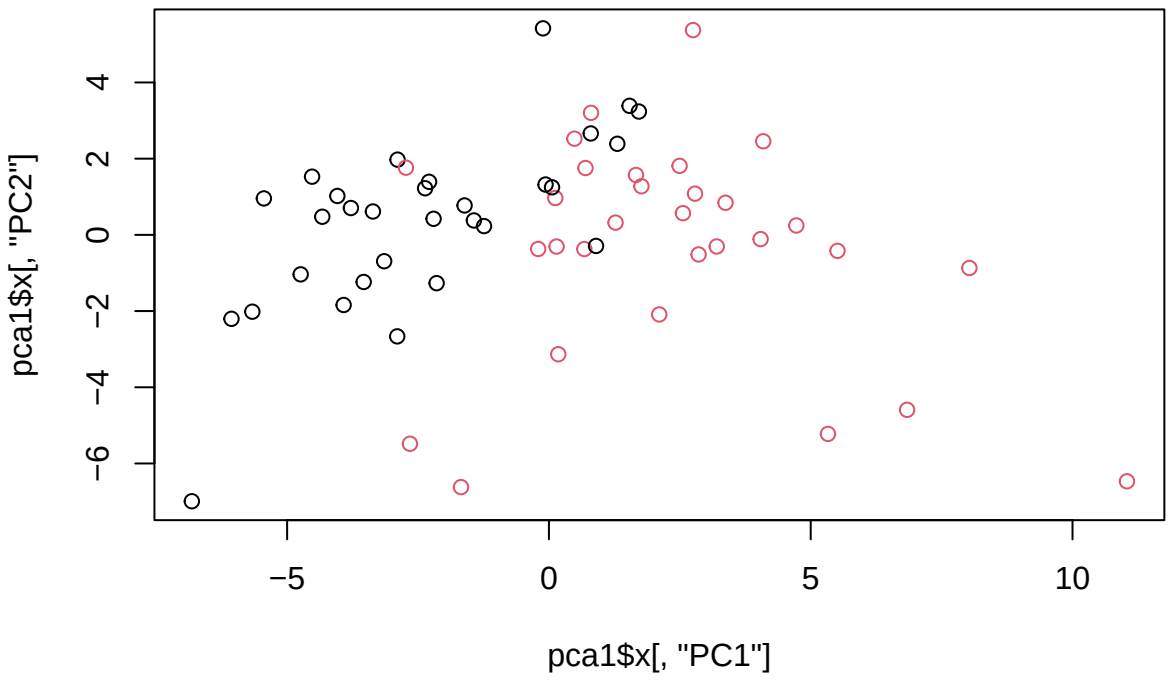
```
#Write your code here
variança<-pca1$sdev^2
barplot(variança)
```



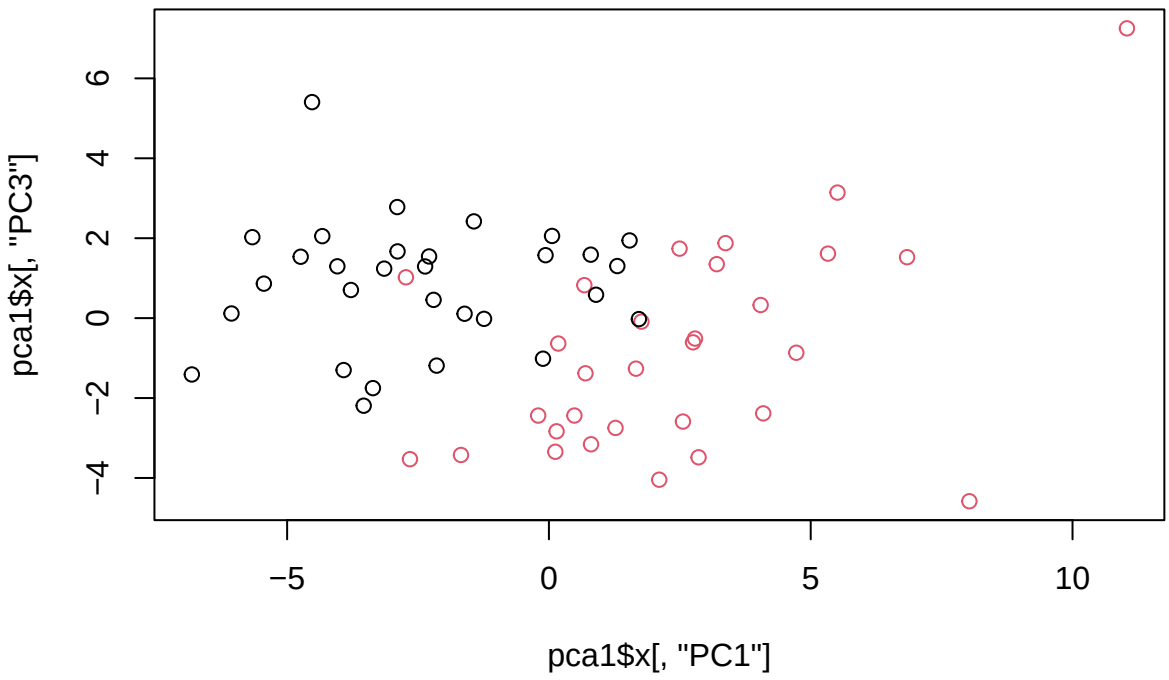
How many PC's are necessary to represent the data? Produce several scatters plots comparing PC1 vs. PC2, PC1 vs. PC3, PC2 vs PC3 and so on to demonstrate which PC's can be discarded to distinguish between "control" and "T2DM_basal" groups. The points of these plots must be colored according to the grouping. pc1 menys relacio amb pc3

```
#Write your code here

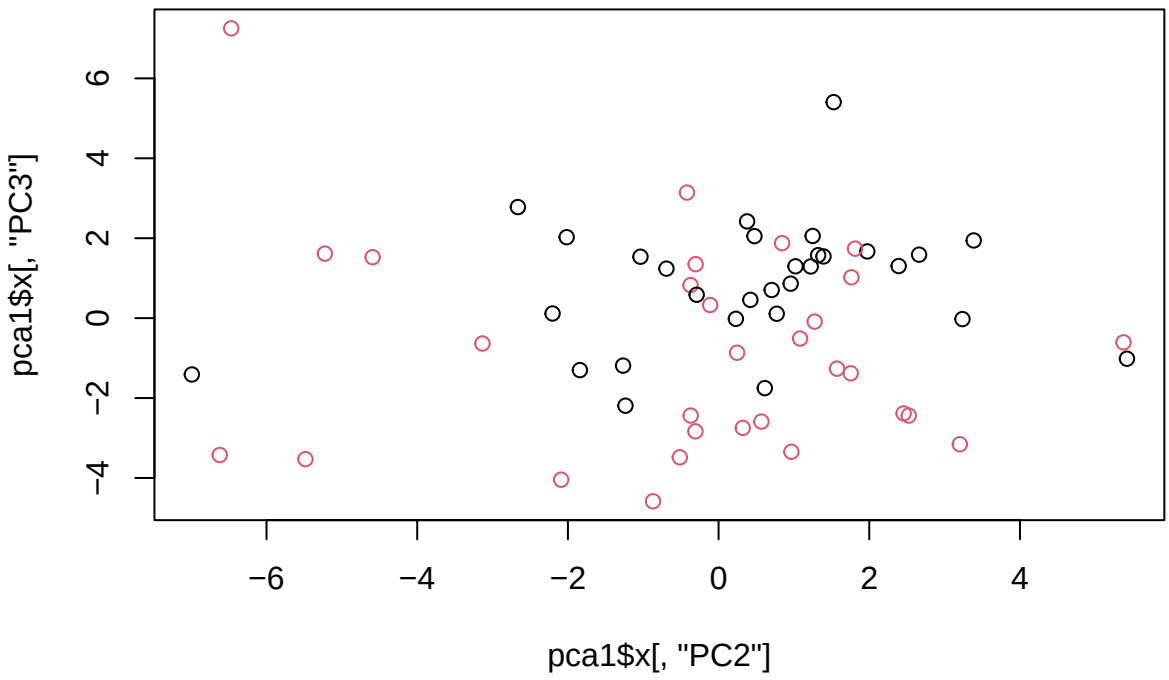
plot(pca1$x[,"PC1"],pca1$x[,"PC2"],col=as.factor(df1$grupo))
```




```
plot(pca1$x[,"PC1"],pca1$x[,"PC3"],col=as.factor(df1$grupo))
```




```
plot(pca1$x[,"PC2"],pca1$x[,"PC3"],col=as.factor(df1$grupo))
```



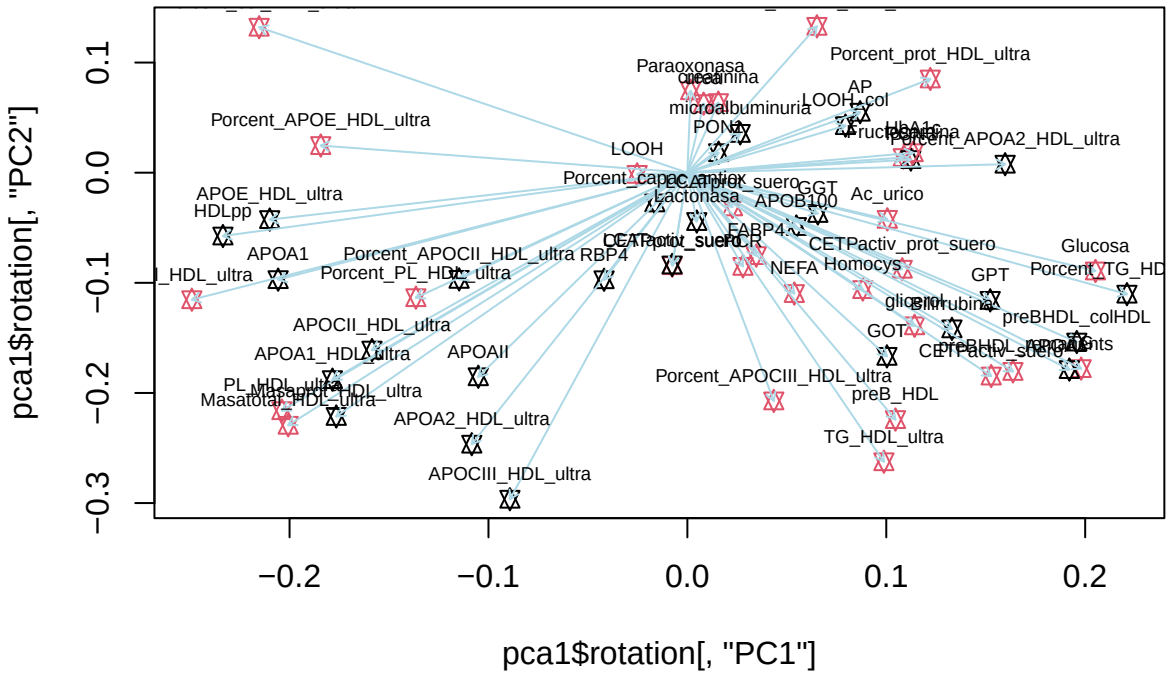
3-Loading analysis

Create the loadings plot for PC1 and PC2.

```
#Write your code here



plot(pca1$rotation[,"PC1"],pca1$rotation[,"PC2"],col=as.factor(df1$grupo),pch=11)
arrows(0,0,pca1$rotation[,"PC1"],pca1$rotation[,"PC2"],length=0.05,
       col="light blue")
text(pca1$rotation[,"PC1"],pca1$rotation[,"PC2"],labels = rownames
     (pca1$rotation),pos=3,cex=0.55)
```



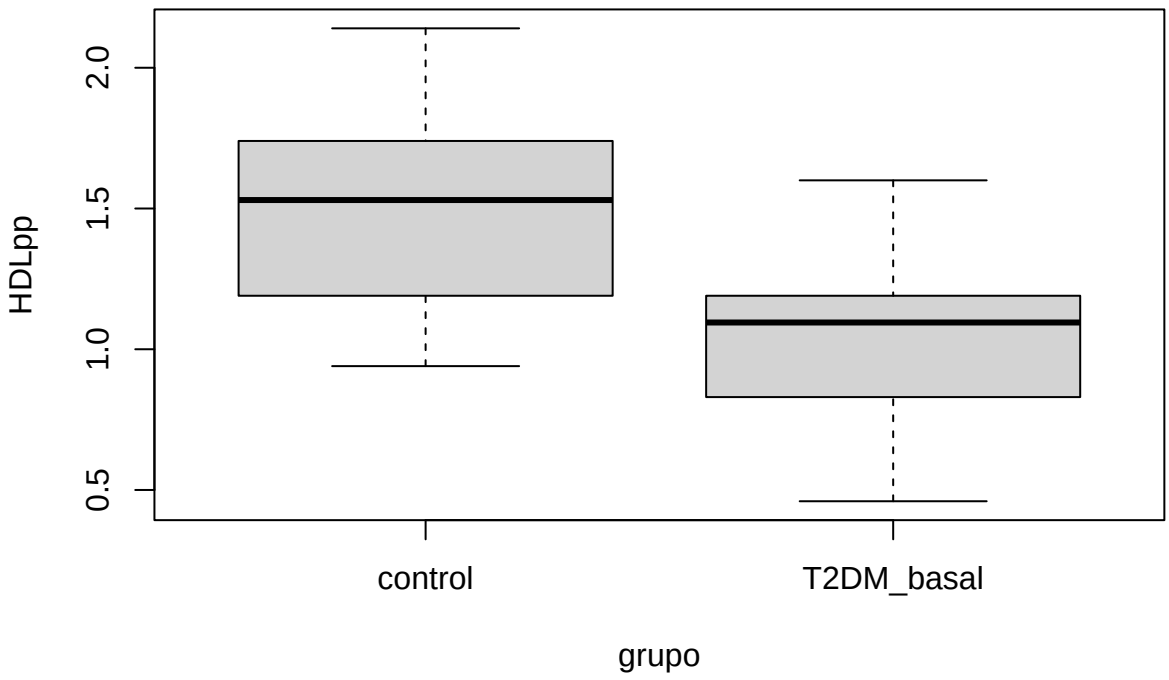
Looking at the PC1 vs. PC2 scatter plot and to the Loadings plot, select three variables with the following properties:

- An up-regulated variable in the "T2DM_basal" group -> HDLpp
- A down-regulated variable in the "T2DM_basal" group -> glucosa
- A variable that is meaningless when the two groups are compared -> Lactonasa

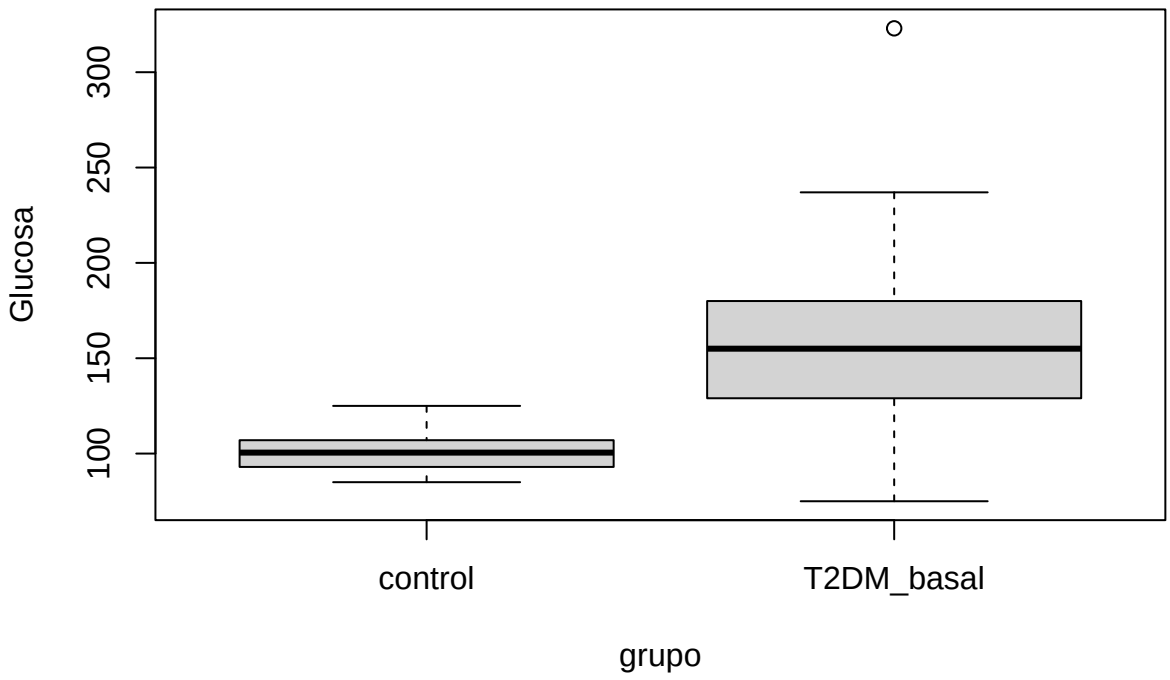
Finally, create a boxplot for each one of the three selected variables. Do they display the expected behavior?

```
#Write your code here

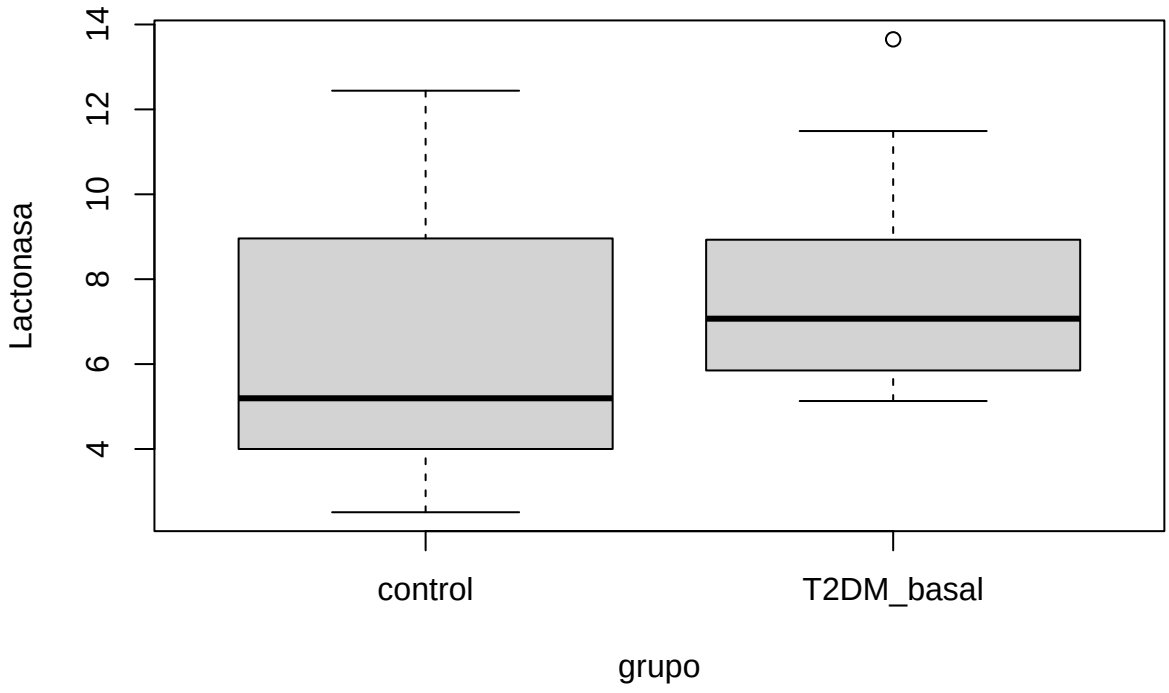
boxplot(HDLpp ~ grupo, data=df1)
```




```
boxplot(Glucosa ~ grupo, data=df1)
```




```
boxplot(Lactonasa ~ grupo, data=df1)
```

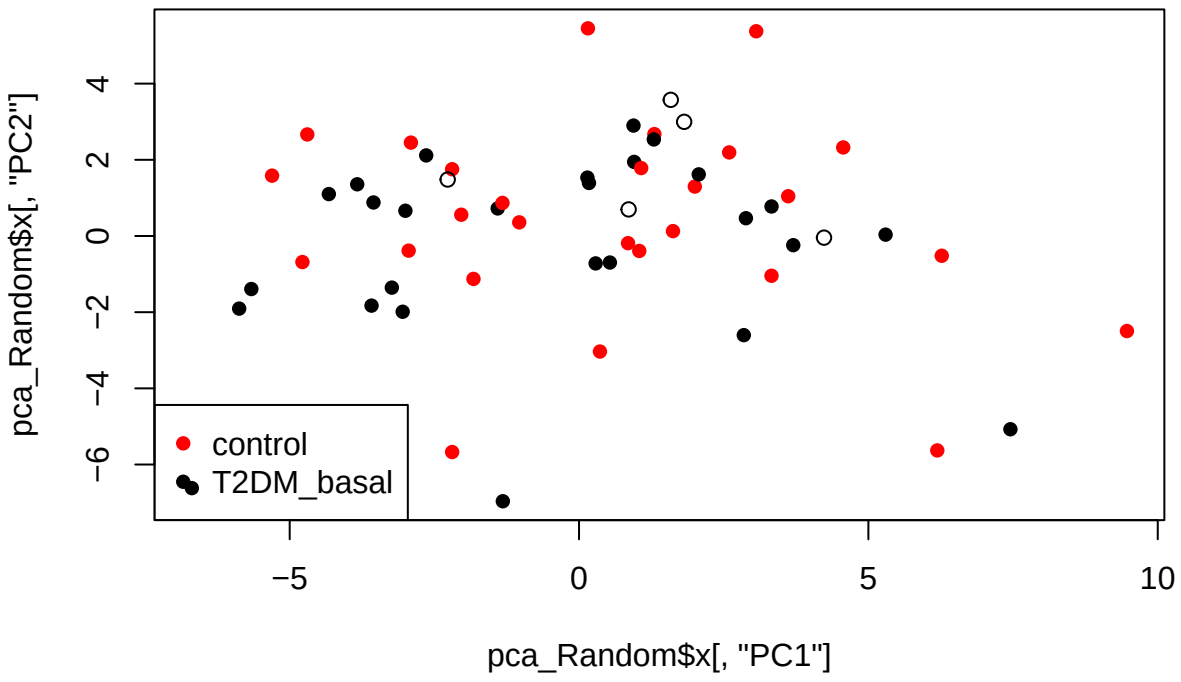


4-PCA as a machine learning algorithm

PCA can be used as a machine learning algorithm to predict the outcome of a new observation. We are now going to simulate some predictions with PCA:

- Select randomly the 90 % of the observations as training observations.
- Create a PCA model using only the training observations.
- Apply the PCA model to the remaining 10 % observations (test observations). Use the R's predict() function to accomplish it.
- Create a PC1 vs. PC2 scatterplot coloring each data point according to its group ("control" vs. "T2DM_basal") and setting a different point symbols according if it is a training or test observation.

```
#Write your code here
colors<-c("control"="red","T2DM_basal"="black")
randomObs <- sort(sample( 1:nrow(df1), round(0.9*nrow(df1))))
pca_Random <- prcomp(df1[randomObs,-1], scale. =  T, center = T )
pdc<-predict(pca_Random,df1[-randomObs,-1])
plot(pca_Random$x[,"PC1"],pca_Random$x[,"PC2"],col=colors[df1$grupo],pch=16)
points(pdc[,"PC1"],pdc[,"PC2"], pch=1)
legend("bottomleft",col = colors, legend = c("control", "T2DM_basal"),pch=16)
```



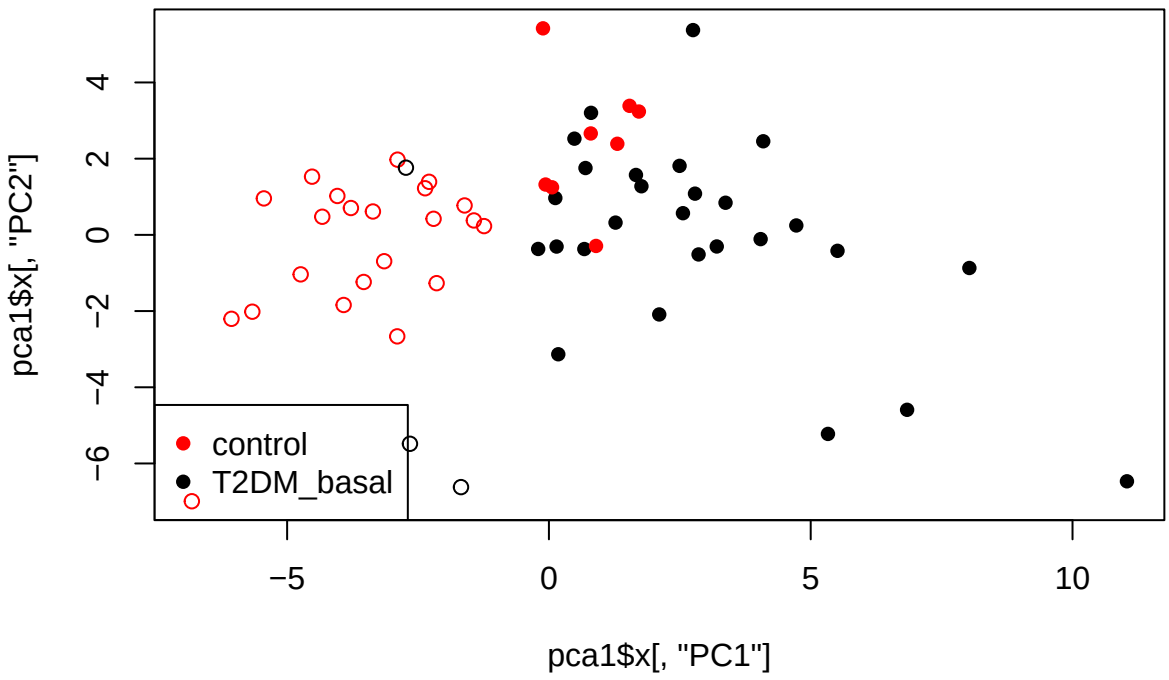
5-K-means clustering

Cluster the observations of the dataset using the k-means clustering algorithm with two clusters. One cluster is expected to be related with "control" group observation and the other with "T2DM_basal" group. Use the R built-in k-means implementation available trough the *kmeans()* function with the parameter *"centers"* set to two clusters. Use the *scale()* function to normalize all variables magnitudes to the same range before clustering.

```
#Write your code here
df_scaled <- scale(df1[,-1])
kmeans_result<-kmeans(df_scaled,centers = 2)
```

The *kmeans()* functions returns a list with several items. Here, we are going to focus on the "cluster" item. This contains a vector with the cluster number which each observation belongs to. Use this information together with the PCA to represent clusters over PC1 vs. PC2 scatter plot. To do so, plot the PC1 vs. PC2 coloring the data points according the grouping (control & T2DM_basal) and use a different data point symbol for each cluster.

```
#Write your code here
colors<-c("control"="red","T2DM_basal"="black")
symbols=c(1,16)
plot(pca1$x[,"PC1"],pca1$x[,"PC2"], col=colors[df1$grupo],
     pch=symbols[kmeans_result$cluster])
legend("bottomleft",col = colors, legend = c("control", "T2DM_basal"),pch=16)
```


pca1$x[, "PC2"]
4
2
0
−2
−4
−6
control
T2DM_basal
−5
0
5
10
pca1$x[, "PC1"]